

■ 1. Flusso di Pubblicazione OTA

#	Fase	Comando / Azione	Cosa succede	File coinvolto
1	Pre-flight Validazione	<code>bash scripts/validate-ota.sh</code>	Controlla CURRENT_OTA_NUMBER, entry ota-updates.json, URL backend, nessun PENDING, hash git valido, live-server guard HTTP 200	lib/ota.ts ota-updates.json app.json
2	Export bundle (Stage 1)	<code>bash scripts/publish-ota.sh export "messaggio"</code>	Bump CURRENT_OTA_NUMBER, aggiunge entry pending in ota-updates.json, esegue npx expo export --platform android, verifica marker nel .hbc	lib/ota.ts ota-updates.json dist-ota/
3	Verifica bundle	(automatico nello Stage 1)	Cerca CURRENT_OTA_NUMBER=N nel binario .hbc con grep -oa. Blocca se non trovato o se il numero non corrisponde	dist-ota/_expo/static/js/android/*.hbc
4	Upload & Crea release (Stage 2)	<code>bash scripts/publish-ota.sh publish</code>	Login admin, upload bundle su Object Storage (private/ota/), POST /api/admin/ota/create con bundleUrl + metadata	server/routes.ts /api/admin/ota/create
5	Promozione slot stable	(automatico nello Stage 2)	POST /api/admin/ota/:id/assign-slot con slot=stable. Aggiorna ota-updates.json (status→published). Tutti i client ricevono l'OTA al prossimo check	/api/admin/ota/:id/assign-slot
6	Rollback (se necessario)	<code>bash scripts/rollback-ota.sh</code>	Login admin, POST /api/admin/ota/:releaseId/publish sull'entry storica, aggiorna lib/ota.ts e ota-updates.json (target→published, corrente→rolled-back)	scripts/rollback-ota.sh ota-updates.json lib/ota.ts

■ 2. Trigger controllo aggiornamenti (client) 3. Logica client

Trigger	Sorgente	Cooldown	Force?	Componente	Funzione	File
Avvio app	startup	60 s (norm) 5 min (3+ fail)	No	Probe HTTP	Fetch parallelo a /api/expo-updates con timeout 8s per diagnostica rete vs SDK	ota-check.ts
Ritorno in fg	appstate	60 s	No	Deferred Reload	Se app in primo piano al momento del fetch, il riavvio viene differito al prossimo backgrounding (AppState→background)	ota-check.ts
OTA Gate (login/reg)	login register	60 s	No	Device ID (Extra-Params)	Updates.setExtraParamAsync("device-id", id) — persiste nello storage nativo, incluso in ogni chiamata OTA	ota-hardenin g.ts
Manuale (admin panel)	manual	Nessuno	Si	Heartbeat Post-Load	POST /api/ota/heartbeat entro 1500ms dal boot se updateId presente. Resetta i contatori stuck in caso di successo	ota-hardenin g.ts
				Circuit Breaker (Stuck Gate)	isOtaStuck() = true se rollbackCount >= 3 o stuckSessions >= 3. Blocca i check OTA e mostra Recovery UI	ota-stuck.ts
				Recovery UI	Mostra banner all'utente con opzione clearOtaStuckState() per resettare i contatori e riprendere gli aggiornamenti	ota-stuck.ts

■ 4. Backend — Endpoint REST

Metodo	Path	Auth	Funzione
GET	/api/expo-updates	—	Serve il manifest OTA (Expo Protocol v1, multipart/mixed). Slot-routing per device-id via expo-extra-params
POST	/api/admin/ota/create	Admin	Crea una nuova release OTA con bundleUrl, metadata e status=pending
POST	/api/admin/ota/:id/assign-slot	Admin	Promuove la release allo slot=stable. I client iniziano a riceverla al prossimo check
POST	/api/admin/ota/:id/publish	Admin	Usato dal rollback: riattiva una release storica (status→active)
POST	/api/ota/heartbeat	—	Riceve heartbeat post-load dal client (deviceId, releaseId, otaNumber). Conferma che il bundle è attivo
POST	/api/admin/ota-error	—	Telemetria errori OTA (phase, source, errorCode, nativeStack, probe, networkInfo). Fire-and-forget dal client

■ 5. Source of Truth — File chiave

File	Ruolo	Aggiornato da
lib/ota.ts	Contiene CURRENT_OTA_NUMBER (costante TS inclusa nel bundle). Deve corrispondere all'ultima entry in ota-updates.json	publish-ota.sh (Stage 1) rollback-ota.sh
ota-updates.json	Registro storico di tutte le OTA: updateNumber, version, runtimeVersion, releaseId, bundleUrl, status, commitBase, publishedAt	publish-ota.sh (Stage 1+2) rollback-ota.sh
app.json	Contiene expo.runtimeVersion (usato per separare i cicli OTA) e expo.updates.url (endpoint backend custom)	Manuale (solo cambio ciclo)
scripts/publish-ota.sh	Script 2-stage: export (Step A-E) + publish (Step 1-7). Gestisce backup, rollback automatico su errore e state file .local/ota-state.json	Manuale / CI
scripts/validate-ota.sh	Pre-flight: 8 check (import, metodi Updates, CURRENT_OTA_NUMBER, PENDING, commitBase, duplicati, URL backend, live-server guard)	Manuale prima di ogni publish
scripts/rollback-ota.sh	Riattiva una release storica tramite releaseId. Aggiorna ota-updates.json e lib/ota.ts	Manuale in caso di regressione
lib/ota-check.ts	triggerOtaCheck(): orchestrazione check→fetch→reload con cooldown, Circuit Breaker, Probe HTTP, Deferred Reload e telemetria	(runtime client)
lib/ota-hardening.ts	initOtaHardening(): Device ID via Extra-Params, listener rollback runtime, heartbeat post-load dopo 1500ms	(runtime client)
lib/ota-stuck.ts	AsyncStorage: rollbackCount, stuckSessions, lastFetchedId. isOtaStuck() attiva il Circuit Breaker dopo 3 eventi	(runtime client)